Pós Graduação
Data Science e Big Data na Indústria

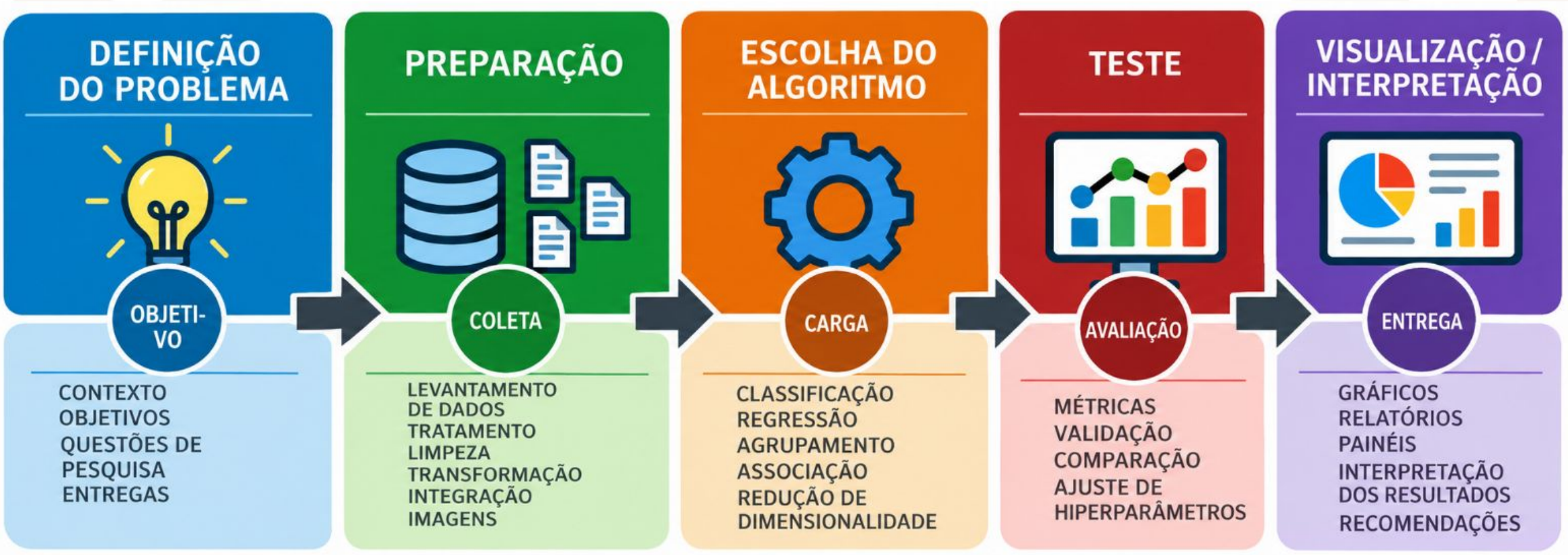
Disciplina: Data Science e Big Data

Aula 05

Prof. Me Daniel Vieira

UNISENAI
Inovação no Ensino Superior para a Indústria

Etapas de um projeto de Data Science

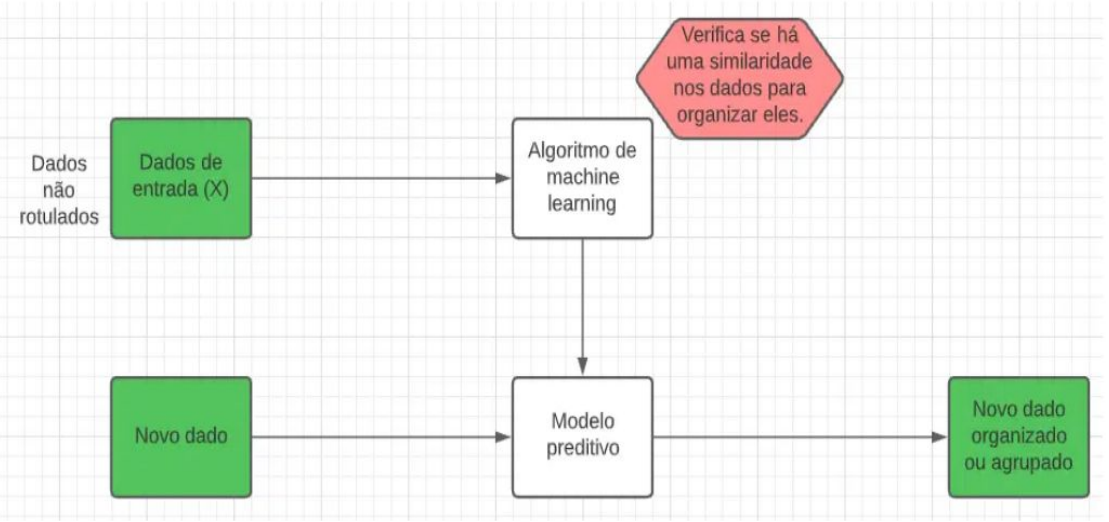


Diferença entre programação tradicional e Machine Learning

- Programação tradicional:
- Dados + Regras programadas → Resultado
- Machine Learning:
- Dados + Resultados conhecidos → Treinamento → Modelo aprendido
- Depois:
- Novos dados + Modelo aprendido → Previsão

Aprendizado não supervisionado

É um algoritmo de aprendizado ocorre de forma independente,
ou seja, o algoritmo busca encontrar relações e semelhanças
entre os dados inseridos no algoritmo.



Aprendizado não supervisionado



Aprendizado não supervisionado

Exemplo 1: Uma indústria deseja analisar o desempenho de suas máquinas com base em dois indicadores: quantidade produzida e consumo de energia elétrica. A empresa coletou dados de nove máquinas, apresentados na tabela abaixo:

	maquina	producao	energia
0	M01	100	50
1	M02	120	55
2	M03	110	52
3	M04	500	200
4	M05	520	210
5	M06	490	195
6	M07	900	400
7	M08	920	410
8	M09	880	390

Aprendizado não supervisionado

Exemplo 1: Desenvolva um programa em Python utilizando as bibliotecas Pandas, Scikit-learn e Matplotlib para realizar as seguintes tarefas:

1. Crie um DataFrame com os dados de produção e consumo de energia das nove máquinas.
2. Utilize o `StandardScaler` para padronizar as variáveis antes de aplicar o algoritmo.
3. Implemente o algoritmo `KMeans`, considerando inicialmente **3 agrupamentos (K = 3)**.
4. Adicione ao DataFrame uma coluna chamada `grupo`, contendo o agrupamento atribuído a cada máquina.
5. Gere um gráfico de dispersão (`plt.scatter`) para visualizar os agrupamentos, utilizando produção no eixo X e consumo de energia no eixo Y.

Aprendizado não supervisionado

Exemplo 1: **Questões para análise**

a) Quantos grupos de máquinas foram identificados pelo algoritmo?

b) Quais máquinas apresentam características semelhantes de produção e consumo energético?

c) Qual grupo apresenta maior produção e maior consumo de energia?

d) Uma máquina que apresenta maior produção necessariamente possui melhor eficiência energética? Justifique.

e) Por que é importante padronizar os dados antes de aplicar o K-Means?

Desafio adicional: Utilize o método do cotovelo (*Elbow Method*) para avaliar se K = 3 é uma escolha adequada para esse conjunto de dados. Considere os valores de K entre 1 e 8.

Aprendizado não supervisionado - Código

```
1
2 # Importa a biblioteca Pandas para criar e manipular tabelas
3 import pandas as pd
4
5 # Importa o Matplotlib para criar gráficos
6 import matplotlib.pyplot as plt
7
8 # Importa o algoritmo K-Means para realizar agrupamentos
9 from sklearn.cluster import KMeans
10
11 # Importa o StandardScaler para padronizar os dados
12 from sklearn.preprocessing import StandardScaler
```

Aprendizado não supervisionado - Código

```
# ========================================
# 1. CRIAÇÃO DO CONJUNTO DE DADOS
# ========================================

# Dados coletados de nove máquinas industriais
# Cada máquina possui informações de produção e
# consumo de energia elétrica

dados = {
    "maquina": ["M01", "M02", "M03",
                "M04", "M05", "M06",
                "M07", "M08", "M09"],

    "producao": [100, 120, 110, 500, 520, 490,
                 900, 920, 880],

    "energia": [50, 55, 52, 200, 210, 195,
                400, 410, 390]
}

# Converte o dicionário em um DataFrame
df = pd.DataFrame(dados)

# Exibe os dados originais
print("Dados das máquinas:")
print(df)
```

Aprendizado não supervisionado - Código

```
# ========================================
# 2. PADRONIZAÇÃO DOS DADOS
# ========================================

# O K-Means utiliza distâncias para agrupar os dados
# Por isso, variáveis com escalas diferentes podem
# influenciar de maneira desigual os agrupamentos

# Cria o objeto responsável pela padronização
scaler = StandardScaler()

# Seleciona as variáveis utilizadas pelo K-Means
# fit_transform():
# fit: calcula a média e o desvio-padrão
# transform: padroniza os valores
#
# Após a padronização:
# Média aproximadamente igual a 0
# Desvio-padrão igual a 1

X = scaler.fit_transform(df[["producao", "energia"]])

# Exibe os dados padronizados
print("\nDados padronizados:")
print(X)

```

Aprendizado não supervisionado - Código

```
28 # ==========================================
29 # 3. CRIAÇÃO DO MODELO K-MEANS
30 # ==========================================
31
32 # Cria o algoritmo de agrupamento K-Means
33 modelo = KMeans(
34     n_clusters=3,    # Quantidade de grupos desejados (K=3)
35     random_state=42, # Permite reproduzir os resultados
36     n_init=10         # Executa o K-Means 10 vezes com
37                       # diferentes inicializações e
38                       # seleciona o resultado de menor inércia
39 )
40
41
42 # ==========================================
43 # 4. TREINAMENTO E IDENTIFICAÇÃO DOS GRUPOS
44 # ==========================================
45
46 # fit_predict(X):
47 # fit: treina o algoritmo utilizando os dados padronizados
48 # predict: identifica a qual grupo cada máquina pertence
49 #
50 # Os grupos são identificados por números: 0, 1 e 2
51 # Esses números não representam uma ordem de desempenho
```

Aprendizado não supervisionado - Código

```
42 # ========================================
43 # 4. TREINAMENTO E IDENTIFICAÇÃO DOS GRUPOS
44 # ========================================
45
46 # fit_predict(X):
47 # fit: treina o algoritmo utilizando os dados padronizados
48 # predict: identifica a qual grupo cada máquina pertence
49 #
50 # Os grupos são identificados por números: 0, 1 e 2
51 # Esses números não representam uma ordem de desempenho
52
53 df["grupo"] = modelo.fit_predict(X)
54
```

Aprendizado não supervisionado - Código

```
# ========================================
# 5. CÁLCULO DA EFICIÊNCIA ENERGÉTICA
# ========================================

# A eficiência energética indica quantas unidades
# são produzidas para cada kWh consumido
#
# Fórmula:
# Eficiência = Produção / Energia
#
# Quanto maior o resultado, maior a quantidade
# produzida por unidade de energia consumida

df["eficiencia"] = df["producao"] / df["energia"]

# Exibe as máquinas organizadas por grupo
print("\nResultado dos agrupamentos:")
print(df.sort_values("grupo"))


```

Aprendizado não supervisionado - Código

```
76 # ========================================
77 # 6. VISUALIZAÇÃO DOS AGRUPAMENTOS
78 # ========================================
79
80 plt.figure(figsize=(10, 8))
81
82 plt.scatter(
83     df["producao"],
84     df["energia"],
85     c=df["grupo"],
86     cmap="viridis",
87     s=120
88 )
89
```

Aprendizado não supervisionado - Código

```
90 # Deslocamentos manuais para evitar sobreposição dos textos
91 deslocamentos = {
92     "M01": (8, 5),
93     "M02": (8, 10),
94     "M03": (8, -10),
95     "M04": (8, 5),
96     "M05": (8, 10),
97     "M06": (8, -10),
98     "M07": (8, 5),
99     "M08": (8, 10),
100    "M09": (8, -10)
101 }
102
```

Aprendizado não supervisionado - Código

```
103 # Adiciona o nome de cada máquina com deslocamento
104 for i in range(len(df)):
105     maquina = df.loc[i, "maquina"]
106     x = df.loc[i, "producao"]
107     y = df.loc[i, "energia"]
108 
109     dx, dy = deslocamentos[maquina]
110 
111     plt.annotate(
112         maquina,
113         (x, y),
114         xytext=(dx, dy),
115         textcoords="offset points",
116         fontsize=10,
117         bbox=dict(boxstyle="round,pad=0.2", fc="white", alpha=0.7)
118     )
119 
120 plt.xlabel("Produção (unidades)")
121 plt.ylabel("Energia (kWh)")
122 plt.title("Agrupamento de Máquinas - K-Means")
123 plt.grid(alpha=0.3)
124 plt.show()
```

Aprendizado não supervisionado - Código

```
125 # ========================================
126 # 7. MÉTODO DO COTOVELO (ELBOW METHOD)
127 # ========================================
128
129 # O método do cotovelo ajuda a avaliar
130 # a quantidade adequada de grupos (K)
131 #
132 # Ele utiliza a inércia (WCSS), que representa
133 # a soma das distâncias quadráticas dos pontos
134 # aos centroides de seus respectivos grupos
135 #
136 # Quanto menor a inércia, mais próximos os pontos
137 # estão dos centroides de seus grupos
138
139 # Cria uma lista vazia para armazenar as inércias
140 inercias = []
141
142 # Testa diferentes quantidades de grupos
143 # range(1, 9) gera valores de K entre 1 e 8
144 for k in range(1, 9):
```

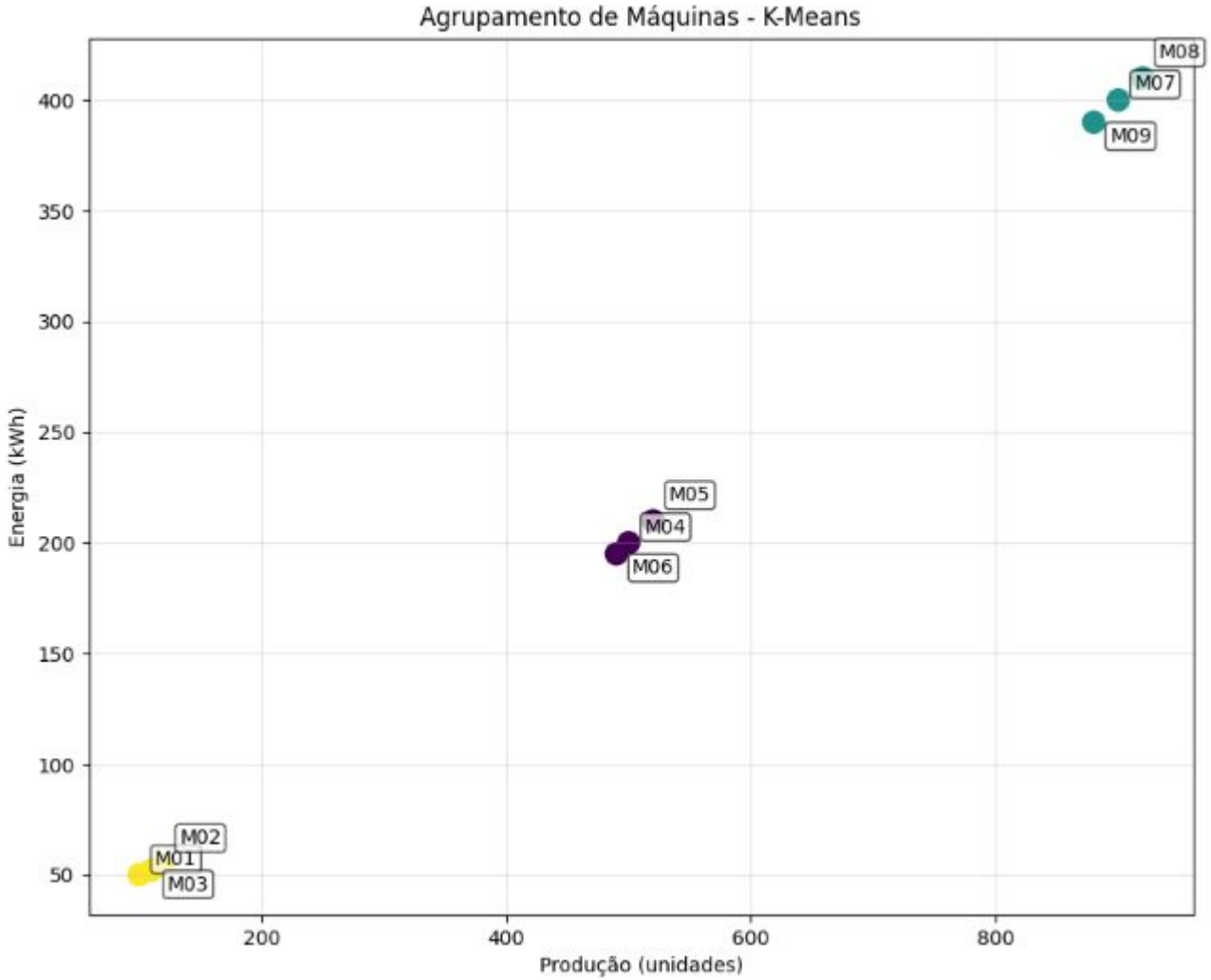
Aprendizado não supervisionado - Código

```
142 # Testa diferentes quantidades de grupos
143 # range(1, 9) gera valores de K entre 1 e 8
144 for k in range(1, 9):
145 
146     # Cria um novo modelo K-Means para cada valor de K
147     modelo_k = KMeans(
148         n_clusters=k,      # Quantidade de grupos testada
149         random_state=42,   # Mantém a reprodutibilidade
150         n_init=10          # Testa 10 inicializações
151     )
152 
153     # Treina o modelo com os dados padronizados
154     modelo_k.fit(X)
155 
156     # inertia_ retorna a inércia do modelo treinado
157     # append() adiciona esse valor à lista
158     inercias.append(modelo_k.inertia_)
159 
160 
161 
162 
```

Aprendizado não supervisionado - Código

```
Resultado dos agrupamentos:
  maquina  producao  energia  grupo  eficiencia
3     M04       500      200      0    2.500000
4     M05       520      210      0    2.476190
5     M06       490      195      0    2.512821
6     M07       900      400      1    2.250000
7     M08       920      410      1    2.243902
8     M09       880      390      1    2.256410
0     M01       100       50      2    2.000000
2     M03       110       52      2    2.115385
1     M02       120       55      2    2.181818
```

Aprendizado não supervisionado - Código



Aprendizado não supervisionado - Código

```python
# ========================================
# 8. GRÁFICO DO MÉTODO DO COTOVELO
# ========================================

# Define o tamanho do gráfico
plt.figure(figsize=(8, 5))

# Cria um gráfico de linha
# Eixo X: quantidade de grupos (K)
# Eixo Y: valor da inércia
plt.plot(
    range(1, 9),
    inercias,
    marker="o"      # Exibe um marcador em cada valor de K
)

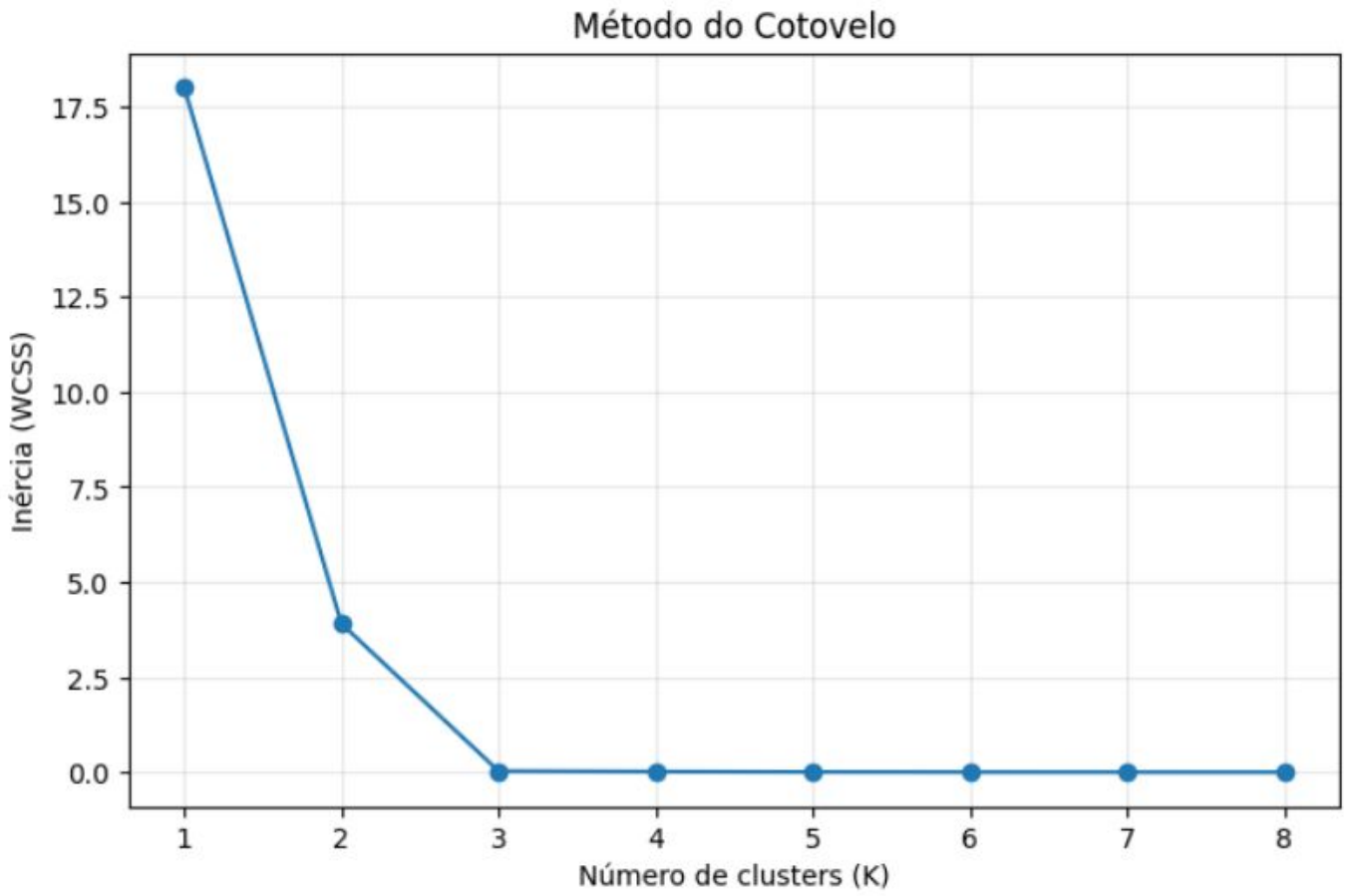
# Define os nomes dos eixos
plt.xlabel("Número de clusters (K)")
plt.ylabel("Inércia (WCSS)")

# Define o título do gráfico
plt.title("Método do Cotovelo")

# Exibe todos os valores de K no eixo X
plt.xticks(range(1, 9))

# Adiciona uma grade
plt.grid(alpha=0.3)

# Exibe o gráfico
plt.show()
```



Aprendizado não supervisionado - Inertia

WCSS – Within-Cluster Sum of Squares

O WCSS (Soma dos Quadrados Dentro dos Clusters) é uma métrica utilizada no algoritmo K-Means para medir o grau de proximidade entre os pontos de cada grupo e seu respectivo centroide.

O WCSS calcula a soma dos quadrados das distâncias entre cada ponto e o centroide do cluster ao qual pertence.

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Em que:

- K: número de clusters.
- C_i: conjunto de pontos do cluster i.
- x: ponto pertencente ao cluster.
- μ_i: centroide do cluster.

Aprendizado não supervisionado - Inertia

Em que:

- K: número de clusters.
- C_i: conjunto de pontos pertencentes ao cluster i.
- x: ponto de dados.
- μ_i: centroide do cluster i.
- $\|x - \mu_i\|^2$: distância euclidiana ao quadrado entre o ponto e o centroide.

Relação com o método do cotovelo

O método do cotovelo utiliza o WCSS para determinar uma quantidade adequada de clusters.

- WCSS alto: os pontos estão mais distantes dos centroides, indicando maior dispersão dentro dos grupos.
- WCSS baixo: os pontos estão mais próximos dos centroides, indicando grupos mais compactos.
- Ponto do cotovelo: região do gráfico em que aumentar o número de clusters deixa de produzir reduções expressivas no WCSS.

Aprendizado não supervisionado - Inertia

Quanto menor a inércia, mais próximos os pontos estão de seus centroides. Entretanto, aumentar K sempre reduz ou mantém a inércia, por isso buscamos o ponto em que a redução começa a perder

Aprendizado não supervisionado Exercícios

Exercício 1 – Agrupamento de Viagens utilizando K-Means

Uma empresa de transportes deseja analisar o perfil de suas viagens para identificar padrões de deslocamento e auxiliar no planejamento logístico.

Para isso, foram coletados dados referentes à distância percorrida (km) e ao tempo de viagem (horas) de 12 viagens realizadas.

A empresa deseja utilizar o algoritmo K-Means para identificar grupos de viagens com características semelhantes, sem definir previamente a categoria de cada viagem.

Aprendizado não supervisionado Exercício 1

Viagem	Distância (km)	Tempo (horas)
V01	15	0,5
V02	20	0,7
V03	18	0,6
V04	25	0,8
V05	150	2,5
V06	180	3,0
V07	160	2,8
V08	200	3,5
V09	650	9,0
V10	700	10,0
V11	680	9,5
V12	750	11,0

Aprendizado não supervisionado - Exercício 1

Desenvolva um programa em Python utilizando Pandas, Scikit-learn e Matplotlib para:

1. Criar um DataFrame contendo os dados das viagens.
2. Selecionar as variáveis `distancia` e `tempo`.
3. Padronizar os dados utilizando `StandardScaler`.
4. Aplicar o algoritmo `KMeans` considerando **K = 3**.
5. Adicionar ao DataFrame uma coluna chamada `grupo`, identificando o agrupamento de cada viagem.
6. Criar um gráfico de dispersão (`plt.scatter`) com distância no eixo X e tempo no eixo Y, diferenciando os grupos por cores.
7. Utilizar o método do cotovelo para analisar a escolha do número de agrupamentos.

Aprendizado não supervisionado - Exercício 1

Questões para análise

a) Quantos grupos de viagens foram identificados?

b) Quais viagens apresentam características semelhantes?

c) Qual grupo apresenta as maiores distâncias e os maiores tempos de deslocamento?

d) Como esses agrupamentos podem auxiliar no planejamento logístico da empresa?

e) O método do cotovelo indica que K = 3 é uma escolha adequada? Justifique.

Aprendizado não supervisionado - Exercício 1

Entregar através do Link:

Exercício 1

Aprendizado não supervisionado - Exercício 2

Exercício 2 – Agrupamento de Máquinas Industriais com Vibração e Corrente Elétrica

Uma indústria utiliza sensores para monitorar as condições operacionais de suas máquinas. Entre os parâmetros monitorados estão a vibração mecânica (mm/s) e a corrente elétrica consumida (A).

A equipe de manutenção deseja identificar máquinas com comportamentos semelhantes e verificar a existência de grupos que apresentem valores elevados dessas variáveis.

Como não existem classificações previamente definidas, será utilizado o algoritmo K-Means para identificar diferentes perfis operacionais.

Aprendizado não supervisionado - Exercício 2

Base de dados

Máquina	Vibração (mm/s)	Corrente (A)
M01	1,2	5,0
M02	1,5	5,5
M03	1,3	5,2
M04	1,7	6,0
M05	3,5	10,0
M06	3,8	11,0
M07	4,0	10,5
M08	4,2	12,0
M09	7,5	18,0
M10	8,0	19,0
M11	8,5	20,0
M12	9,0	21,0

Aprendizado não supervisionado - Exercício 2

Desenvolva um programa em Python utilizando Pandas, Scikit-learn e Matplotlib para:

1. Criar um DataFrame contendo os dados das máquinas.
2. Selecionar as variáveis `vibracao` e `corrente`.
3. Padronizar os dados utilizando `StandardScaler`.
4. Implementar o algoritmo `KMeans` considerando **K = 3**.
5. Adicionar ao DataFrame uma coluna chamada `grupo`.
6. Criar um gráfico de dispersão (`plt.scatter`) com vibração no eixo X e corrente elétrica no eixo Y, utilizando cores diferentes para representar os grupos.
7. Aplicar o método do cotovelo para avaliar a escolha do número de agrupamentos.

Aprendizado não supervisionado - Exercício 2

Questões para análise

a) Quantos grupos de máquinas foram identificados?

b) Quais máquinas apresentam os menores valores de vibração e corrente elétrica?

c) Qual grupo apresenta os maiores valores dessas variáveis?

d) Os agrupamentos podem ajudar a equipe de manutenção a identificar máquinas que precisam de inspeção? Explique.

e) Uma máquina com vibração e corrente elevadas necessariamente apresenta uma falha? Justifique.

f) O método do cotovelo confirma K = 3 como uma escolha adequada?

Aprendizado não supervisionado - Exercício 2

Exercício 2

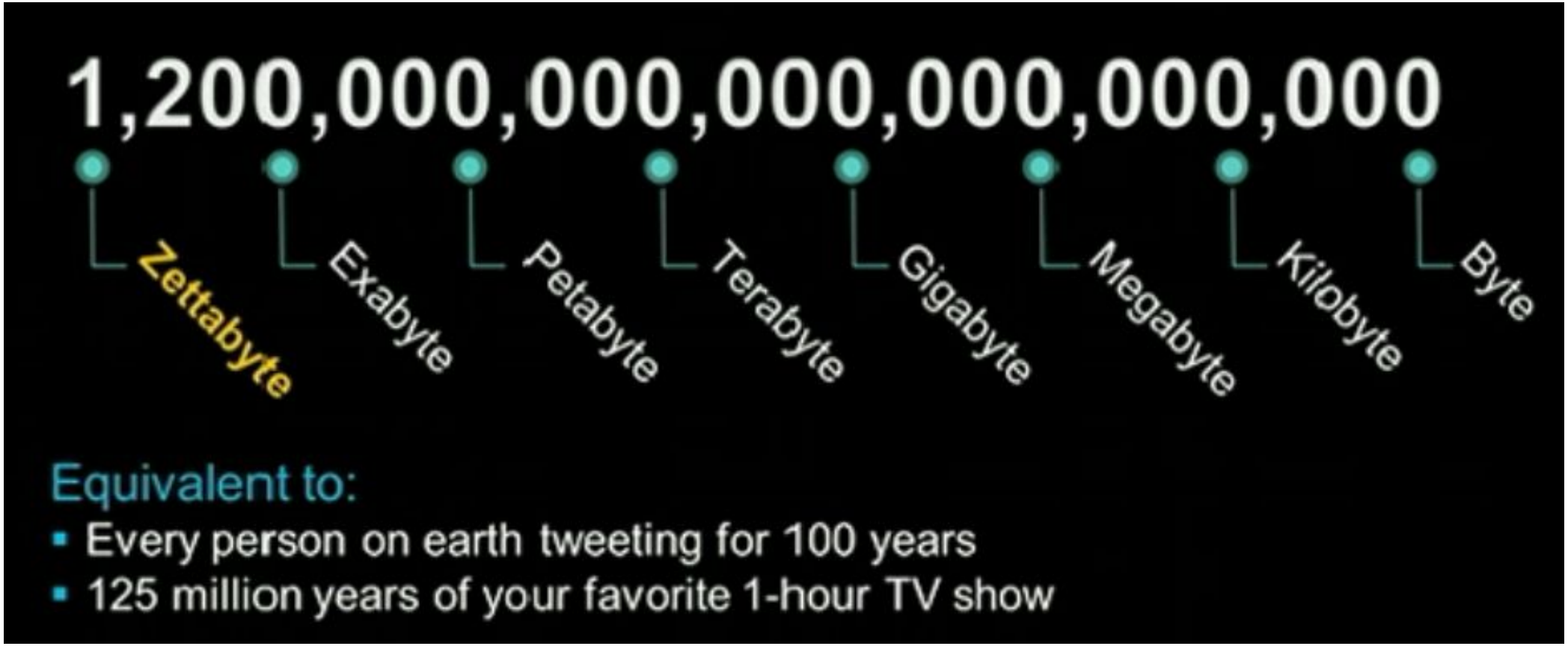
Big Data

Big Data - significa a organização de um grande volume de dados provenientes de diferentes fontes.
Dessa forma a medida que a quantidade de dispositivos aumentam, o volume dos dados aumentam também.
Então é necessário organizar os dados de forma que possa realizar análises e fazer diagnósticos.
Big Data faz referência ao grande volume, variedade e velocidade de dados que demandam formas inovadoras e rentáveis de processamento da informação, para melhor percepção e tomada de decisão.”

Big Data na Indústria



Ordem de grandeza dos dados



Os 5 Vs do Big Data

1. Volume — Quantidade de dados

Refere-se à grande quantidade de dados gerados e armazenados.

Exemplo industrial: uma fábrica com 500 sensores coletando informações de temperatura, corrente elétrica e vibração continuamente.

2. Velocidade — Rapidez dos dados

Refere-se à rapidez com que os dados são gerados, transmitidos e processados.

Exemplo industrial: sensores enviando leituras a cada segundo para detectar condições anormais de funcionamento.

3. Variedade — Diferentes formatos

Os dados podem ser estruturados, semiestruturados ou não estruturados.

Exemplo industrial: planilhas de produção, registros JSON de sensores, imagens de inspeção e vídeos das máquinas.

Os 5 Vs do Big Data

4. Veracidade — Confiabilidade dos dados

Está relacionada à qualidade, precisão e confiabilidade das informações coletadas.

Exemplo industrial: identificar leituras incorretas, valores ausentes ou medições de sensores descalibrados.

5. Valor — Utilidade dos dados

Refere-se à capacidade de transformar os dados em benefícios para a organização.

Exemplo industrial: utilizar modelos de Machine Learning para prever falhas, reduzir desperdícios e otimizar o consumo de energia.

Os 5 Vs do Big Data

4. Veracidade — Confiabilidade dos dados

Está relacionada à qualidade, precisão e confiabilidade das informações coletadas.

Exemplo industrial: identificar leituras incorretas, valores ausentes ou medições de sensores descalibrados.

5. Valor — Utilidade dos dados

Refere-se à capacidade de transformar os dados em benefícios para a organização.

Exemplo industrial: utilizar modelos de Machine Learning para prever falhas, reduzir desperdícios e otimizar o consumo de energia.

Os 5 Vs do Big Data

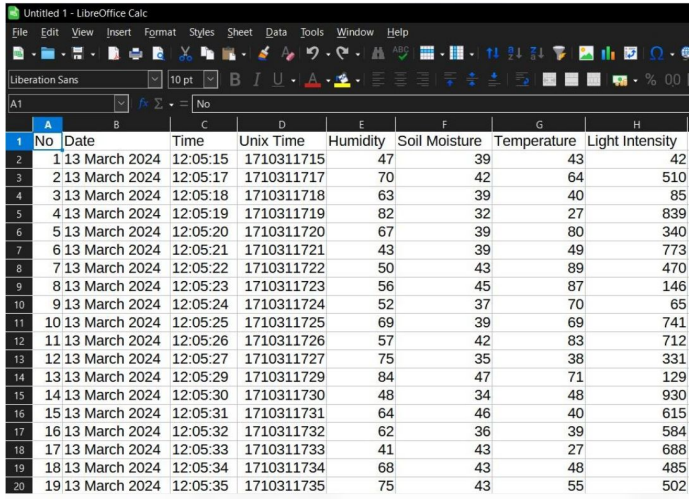
Uma indústria possui 200 máquinas, cada uma equipada com sensores que registram o consumo energético a cada segundo. Os dados são armazenados em uma plataforma e utilizados para identificar desperdícios e prever problemas operacionais.



Atividade

1. Identifique os cinco Vs do Big Data presentes neste cenário.
2. Explique qual dos cinco Vs está associado ao tratamento dos dados que vocês realizaram nas aulas anteriores.

Tipos de dados no Big Data



No	Date	Time	Unix Time	Humidity	Soil Moisture	Temperature	Light Intensity
1	13 March 2024	12:05:15	1710311715	47	39	43	42
2	13 March 2024	12:05:17	1710311717	70	42	64	510
3	13 March 2024	12:05:18	1710311718	63	39	40	85
4	13 March 2024	12:05:19	1710311719	82	32	27	839
5	13 March 2024	12:05:20	1710311720	67	39	80	340
6	13 March 2024	12:05:21	1710311721	43	39	49	773
7	13 March 2024	12:05:22	1710311722	50	43	89	470
8	13 March 2024	12:05:23	1710311723	56	45	87	146
9	13 March 2024	12:05:24	1710311724	52	37	70	65
10	13 March 2024	12:05:25	1710311725	69	39	69	741
11	13 March 2024	12:05:26	1710311726	57	42	83	712
12	13 March 2024	12:05:27	1710311727	75	35	38	331
13	13 March 2024	12:05:29	1710311729	84	47	71	129
14	13 March 2024	12:05:30	1710311730	48	34	48	930
15	13 March 2024	12:05:31	1710311731	64	46	40	615
16	13 March 2024	12:05:32	1710311732	62	36	39	584
17	13 March 2024	12:05:33	1710311733	41	43	27	688
18	13 March 2024	12:05:34	1710311734	68	43	48	485
19	13 March 2024	12:05:35	1710311735	75	43	55	502

1. Dados estruturados

Possuem formato fixo e organização bem definida, geralmente em linhas e colunas. São armazenados frequentemente em bancos de dados relacionais e facilmente consultados e processados.

Exemplos: registros de estoque, vendas, dados financeiros e medições de sensores industriais.

```json
{
  "name": "BitsKingdom",
  "address": "Georgia US",
  "url": "https://www.bitskingdom.com",
  "members": [
    {
      "name": "John Doe",
      "skills": [
        "PHP",
        "JavaScript",
        "CSS"
      ]
    },
    {
      "name": "Jane Doe",
      "skills": [
        "PHP",
        "JavaScript",
        "CSS"
      ]
    }
  ]
}
```

2. Dados semiestruturados

Não seguem necessariamente um esquema fixo, mas possuem elementos organizadores, como tags, chaves e rótulos, que permitem identificar suas informações.

Exemplos: arquivos JSON, XML, documentos HTML, e-mails e registros de aplicações.

Tipos de dados no Big Data



3. Dados não estruturados

Não possuem uma organização tabular ou um esquema predefinido. Geralmente exigem técnicas específicas para extrair informações úteis.

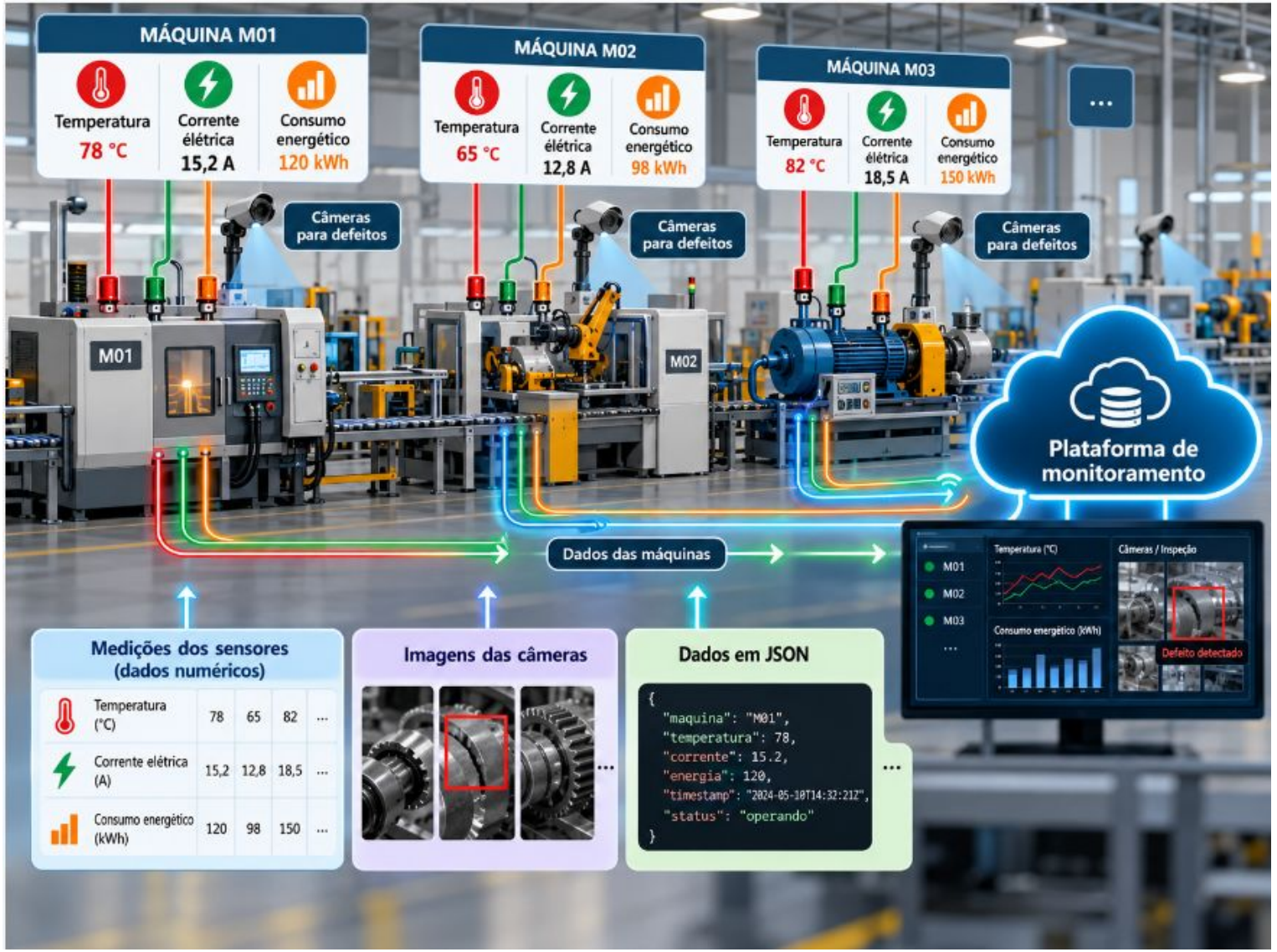
Exemplos: imagens, vídeos, gravações de áudio, documentos de texto e imagens de inspeção industrial.

Tipos de dados no Big Data

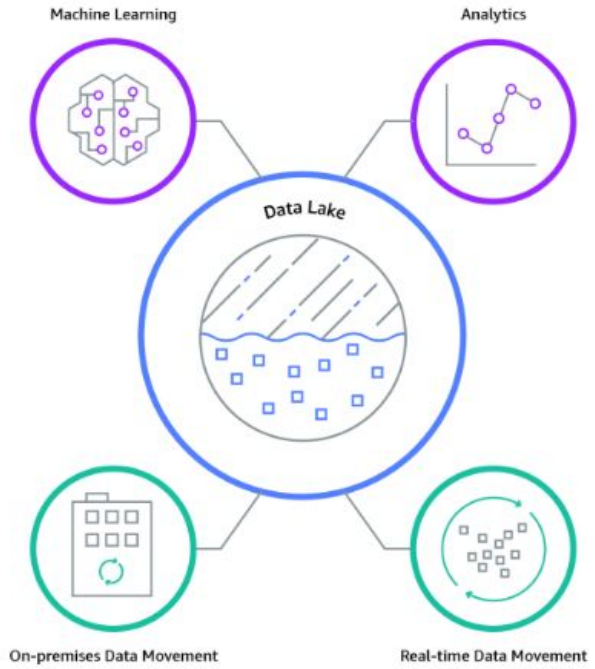
Exemplos aplicados à indústria

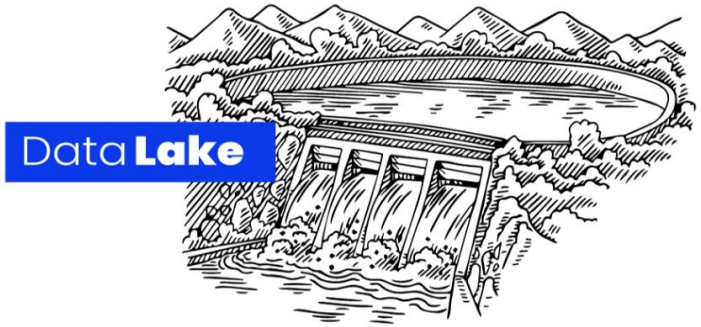
Tipo	Exemplo
Estruturados	Arquivo CSV contendo máquina, temperatura, corrente elétrica e consumo energético
Semiestruturados	Mensagens JSON enviadas por sensores IoT
Não estruturados	Imagens de câmeras utilizadas para identificar defeitos nos equipamentos

Tipos de dados no Big Data



Data Lake





- Ingere dados brutos, **estruturados, semiestruturados** e **não estruturados,** de diversas fontes e em qualquer escala.
- Mantém **centralizados** dados que existiam em silos.
- Conecta-se à ferramentas de **analytics e machine learning.**
- Movimenta os dados em **tempo real.**
- Estruturas mais **complexas, modulares que permitem combinar tecnologias com melhor custo-benefício.**
- Ideais para organizações que buscam criar uma cultura data driven, gerando valor a partir de **engenharia e ciência de dados.**

Data Lake

O Data Lake permite armazenar grandes volumes de dados estruturados, semiestruturados e não estruturados, inclusive em seus formatos originais.

Ferramenta	Empresa / Ecossistema	Aplicação
Azure Data Lake Storage Gen2	Microsoft Azure	Armazenamento de grandes volumes de dados na nuvem
Amazon S3	AWS	Armazenamento de arquivos, logs, imagens e dados de sensores
Google Cloud Storage	Google Cloud	Armazenamento escalável de diferentes formatos
Hadoop HDFS	Apache	Sistema distribuído de armazenamento
Delta Lake	Delta Lake	Camada de tabelas com controle transacional sobre um Data Lake
Microsoft OneLake	Microsoft Fabric	Data Lake unificado para análises corporativas

Data Warehouse



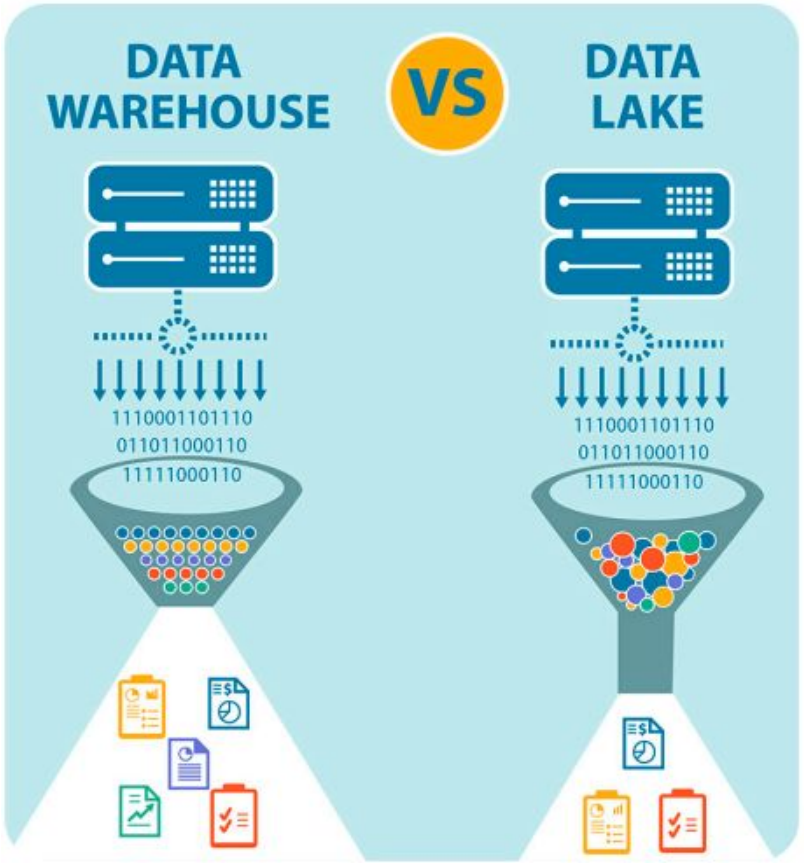
- Armazena de forma **centralizada dados estruturados de bases relacionais e de sistemas,** como CRM, ERPs entre outros.
- Ideais para demandas pontuais de **geração de relatórios para business users.**
- Alta performance com **custo alto.**

Data Warehouse

O Data Warehouse é voltado ao armazenamento de dados integrados, organizados e preparados para consultas analíticas, relatórios e tomada de decisões.

Ferramenta	Empresa / Ecossistema	Aplicação
Amazon Redshift	AWS	Análises de grandes volumes de dados com SQL
Google BigQuery	Google Cloud	Consultas analíticas em grandes conjuntos de dados
Snowflake	Snowflake	Plataforma de dados e análises em nuvem
Microsoft Fabric Data Warehouse	Microsoft	Consultas SQL, relatórios e integração com Power BI
Azure Synapse Analytics	Microsoft Azure	Plataforma de análise e armazenamento analítico
Teradata Vantage	Teradata	Análise corporativa de grandes volumes de dados

Data Warehouse x Data Lake



Adaptado de Grazitti Interactive (2018)

DATA WAREHOUSE

- Dados: processados
- repositório estruturado
- Propósito: específico e pré-definido

Análise em dados processados

DATA LAKE

- Dados: brutos
- estruturados, semi e não-estruturados

Data Warehouse x Data Lake

Característica	Data Lake	Data Warehouse
Dados	Estruturados, semiestruturados e não estruturados	Predominantemente estruturados
Tratamento	Pode armazenar dados brutos	Dados geralmente tratados e integrados
Organização	Flexível	Esquema definido
Utilização	Big Data, IA e Machine Learning	BI, relatórios e análises SQL
Exemplos	Amazon S3, Azure Data Lake	BigQuery, Redshift, Snowflake

Ferramentas Big Data

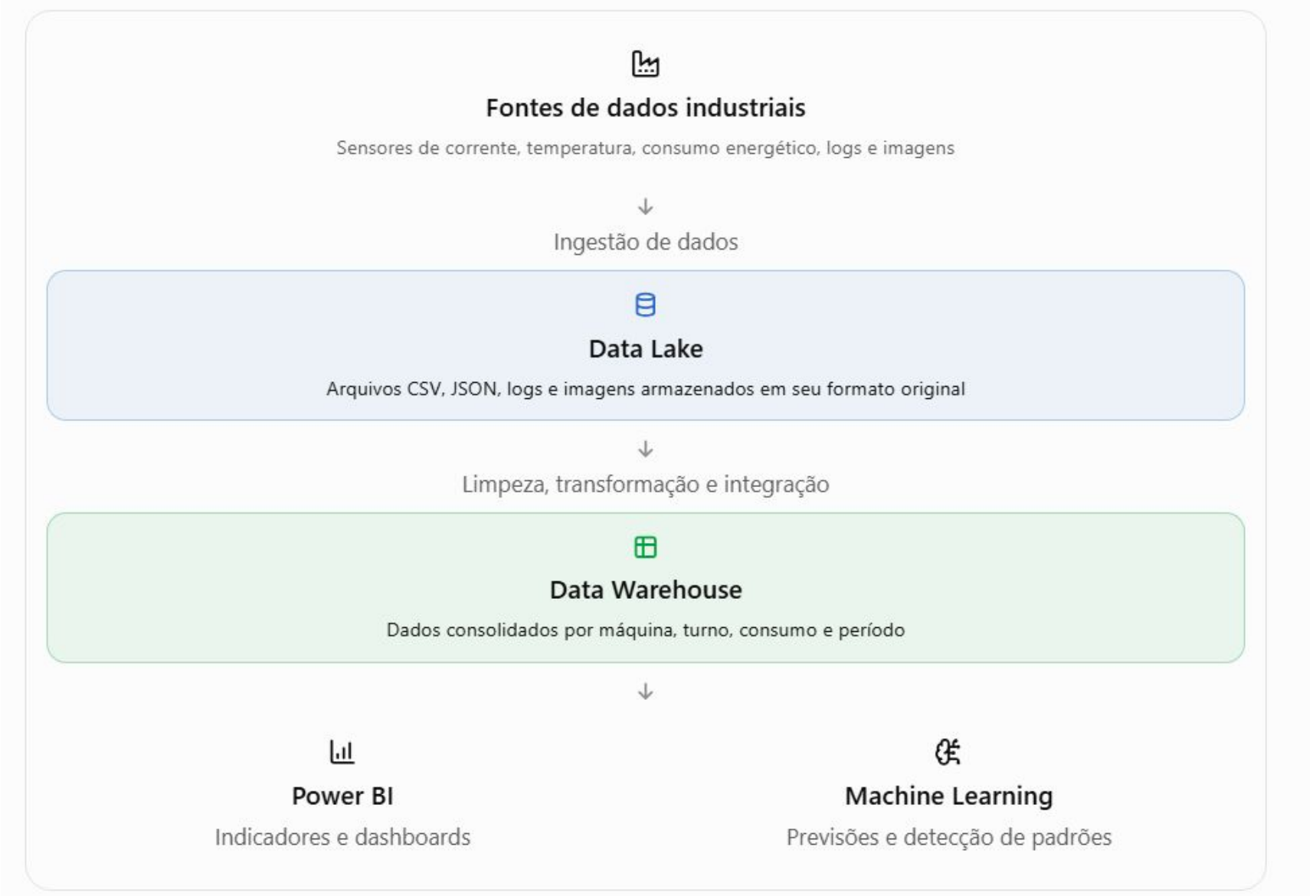
aws

Azure

Google Cloud



Exemplo Indústria consumo energético



Exemplo Indústria consumo energético com Apache Spark

Objetivo: reutilizar a base limpa de consumo energético, gerar indicadores operacionais e discutir processamento em escala. A base original tem 5.000 registros e não exige um cluster. A ampliação final é **artificial e somente pedagógica**.

1. Cenário e perguntas

A indústria deseja avaliar consumo, eficiência por turno e padrões ao longo do tempo.

- Qual turno possui maior consumo médio?
- Qual turno consome menos energia por unidade produzida?
- Como variam consumo e temperatura?
- Como calcular indicadores por mês?
- Por que o Spark permite escalar o processamento?

```
1 %pip -q install pyspark
```

Exemplo Indústria consumo energético com Apache Spark

Esse comando instala a biblioteca PySpark no ambiente do Google Colab.

- %pip: permite instalar bibliotecas diretamente no notebook.
- -q: significa quiet, reduzindo as mensagens de instalação.
- pyspark: biblioteca que permite utilizar o Apache Spark com Python.

Explicação para os alunos: o Pandas normalmente processa os dados na memória de uma única máquina. O Spark foi desenvolvido para processar grandes volumes de dados e pode distribuir o processamento entre vários computadores.

Exemplo Indústria consumo energético com Apache Spark

2. Iniciar Spark em modo local

`local[*]` aproveita os núcleos disponíveis no computador. **Não representa um cluster de máquinas.**

[2]
✓ 16s

```
1 from pyspark.sql import SparkSession, functions as F
2
3 spark = (SparkSession.builder
4          .master("local[*]")
5          .appName("ConsumoEnergeticoIndustrial")
6          .getOrCreate())
7 print("Spark:", spark.version)
```

```
Spark: 4.0.4
```

Exemplo Indústria consumo energético com Apache Spark

A `SparkSession` é o ponto de entrada para trabalhar com dados estruturados no Spark.

Comando	Explicação
`SparkSession.builder`	Inicia a configuração da sessão.
`.master("local[*]")`	Executa o Spark localmente, utilizando os núcleos disponíveis do processador.
`.appName(...)`	Define o nome da aplicação Spark.
`.getOrCreate()`	Reutiliza uma sessão existente ou cria uma nova.
`spark.version`	Mostra a versão do Spark.

Já `functions as F` importa funções de processamento, como soma, média, contagem e correlação, usando o prefixo `F`.

Exemplo Indústria consumo energético com Apache Spark

```
F.avg("consumo_kwh")  # Calcula a média
F.sum("consumo_kwh")  # Calcula a soma
F.count("*")          # Conta os registros
```

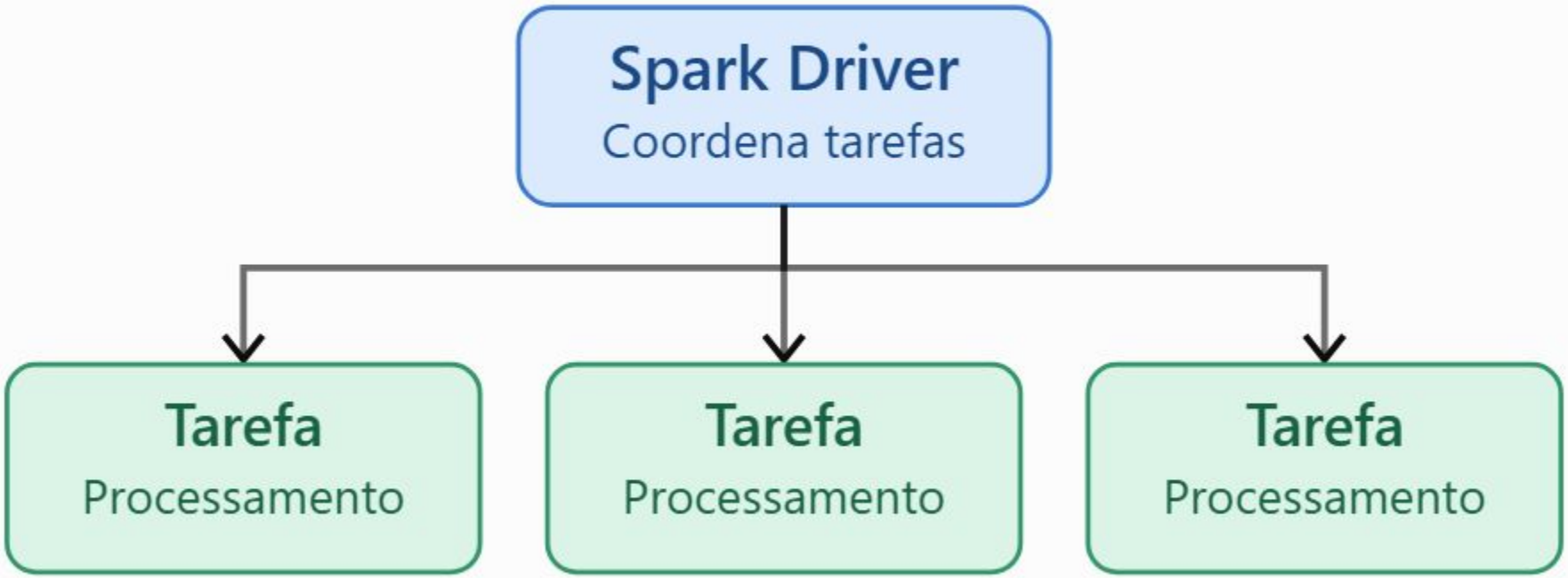
Importa as funções de manipulação e agregação de dados do PySpark.

O F é um apelido que permite chamar funções como:

- F.col(): seleciona uma coluna.
- F.floor(): arredonda um número para baixo até o inteiro mais próximo.
- F.avg(): calcula a média.
- F.round(): arredonda valores.
- F.count(): conta registros.

Exemplo Indústria consumo energético com Apache Spark

O Spark utiliza uma arquitetura que pode envolver um *driver*, responsável por coordenar as operações, e *executors*, responsáveis por executar as tarefas.



Representação conceitual da distribuição de tarefas pelo Spark. No Colab, tudo será executado localmente.

Exemplo Indústria consumo energético com Apache Spark

3. Abrir a base limpa

No Google Colab, faça upload do CSV quando solicitado. Em um ambiente Jupyter local, coloque o CSV na mesma pasta do notebook.

[3] ✓ 6s

```python
from pathlib import Path

arquivo = Path("consumo_energetico_limpa_professor.csv")
if not arquivo.exists():
    try:
        from google.colab import files
        print("Selecione consumo_energetico_limpa_professor.csv")
        files.upload()
    except ImportError:
        raise FileNotFoundError("Coloque o CSV ao lado do notebook e execute novamente.")

df = (spark.read.option("header", True)
      .option("inferSchema", True)
      .csv(str(arquivo)))
df.printSchema()
df.show(5, truncate=False)
```

Exemplo Indústria consumo energético com Apache Spark

Esse trecho verifica se o arquivo CSV está disponível.

- `Path(...)` : representa o caminho do arquivo.
- `arquivo.exists()` : verifica se o arquivo existe.
- `if not` : executa a condição quando o arquivo não existe.
- `files.upload()` : abre a janela para carregar um arquivo no Google Colab.
- `except ImportError` : trata o caso em que o código não está em um ambiente com a biblioteca do Colab.

Exemplo Indústria consumo energético com Apache Spark

```
df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(str(arquivo))
)

df.printSchema()
df.show(5, truncate=False)
```

Aqui, o Spark lê o CSV e cria um DataFrame distribuído.

O comando header=True informa que a primeira linha contém os nomes das colunas.

O inferSchema=True solicita que o Spark identifique automaticamente tipos como inteiro, decimal e texto.

O printSchema() mostra a estrutura dos dados, enquanto show(5) apresenta os primeiros cinco registros.

A diferença importante para os alunos é que o df agora é um DataFrame do Spark, não do Pandas.

Exemplo Indústria consumo energético com Apache Spark

4. Confirmar a qualidade dos dados

Mesmo uma base já limpa deve passar por verificações de entrada no pipeline.

[4] ✓ 4s

```
print("Registros:", df.count())
print("Colunas:", len(df.columns))
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns]).show()
print("Linhas distintas:", df.distinct().count())
```

```
Registros: 5000
Colunas: 7
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+
|timestamp|turno|producao_unidades|temperatura_ambiente_c|maquinas_ativas|horas_operacao_turno|consumo_kwh|
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+
|        0|    0|               0|                    0|             0|                   0|          0|
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+

Linhas distintas: 5000
```

Exemplo Indústria consumo energético com Apache Spark

4. Confirmar a qualidade dos dados

Mesmo uma base já limpa deve passar por verificações de entrada no pipeline.

[4] ✓ 4s

```
print("Registros:", df.count())
print("Colunas:", len(df.columns))
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns]).show()
print("Linhas distintas:", df.distinct().count())
```

```
Registros: 5000
Colunas: 7
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+
|timestamp|turno|producao_unidades|temperatura_ambiente_c|maquinas_ativas|horas_operacao_turno|consumo_kwh|
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+
|        0|    0|               0|                    0|             0|                   0|          0|
+---------+-----+----------------+---------------------+--------------+--------------------+-----------+

Linhas distintas: 5000
```

Exemplo Indústria consumo energético com Apache Spark

5. Criar variáveis para a análise

A intensidade energética por registro é apresentada para inspeção. Para comparar grupos, usaremos a **razão entre somas**, que evita distorções ao tirar a média de razões individuais.

+ Código + Texto

[5] ✓ 0s

```
df = df.withColumn("data_hora", F.to_timestamp("timestamp"))
df = df.withColumn("ano_mes", F.date_format("data_hora", "yyyy-MM"))
df = df.withColumn("kwh_por_unidade_registro",
    F.when(F.col("producao_unidades") > 0, F.col("consumo_kwh") / F.col("producao_unidades")))
df.select("turno", "consumo_kwh", "producao_unidades", "kwh_por_unidade_registro").show(5)
```

```
+-----+-----------+-----------------+------------------------+
|turno|consumo_kwh|producao_unidades|kwh_por_unidade_registro|
+-----+-----------+-----------------+------------------------+
|manha|      679.7|             3139|      0.2165339280025486|
|tarde|     507.48|             2447|     0.20738863914997957|
|noite|     322.41|             1481|     0.21769750168804863|
|manha|     352.18|             1575|      0.2236063492063492|
|tarde|     180.74|              762|     0.23719160104986878|
+-----+-----------+-----------------+------------------------+
only showing top 5 rows
```

Exemplo Indústria consumo energético com Apache Spark

6. Indicadores por turno

[6] ✓ 2s

```
por_turno = (df.groupBy("turno")
    .agg(F.count("*").alias("registros"),
         F.round(F.avg("consumo_kwh"), 2).alias("consumo_medio_kwh"),
         F.round(F.sum("consumo_kwh"), 2).alias("consumo_total_kwh"),
         F.sum("producao_unidades").alias("producao_total"),
         F.round(F.avg("temperatura_ambiente_c"),2).alias("temperatura_media_c"))
    .withColumn("kwh_por_unidade", F.round(F.col("consumo_total_kwh") / F.col("producao_total"), 4))
    .orderBy("kwh_por_unidade"))
por_turno.show(truncate=False)
```

```
+-----+---------+-----------------+-----------------+--------------+------------------+---------------+
|turno|registros|consumo_medio_kwh|consumo_total_kwh|producao_total|temperatura_media_c|kwh_por_unidade|
+-----+---------+-----------------+-----------------+--------------+------------------+---------------+
|noite|1666     |401.66           |669161.45        |3162475       |25.96             |0.2116         |
|tarde|1667     |396.71           |661317.75        |3114914       |26.05             |0.2123         |
|manha|1667     |399.25           |665553.95        |3133212       |26.09             |0.2124         |
+-----+---------+-----------------+-----------------+--------------+------------------+---------------+
```

Exemplo Indústria consumo energético com Apache Spark

7. Consultas com Spark SQL

A classe pode comparar a consulta SQL com os comandos DataFrame.

[7] ✓ 0s

```
df.createOrReplaceTempView("consumo")
spark.sql("""
SELECT turno, COUNT(*) AS registros,
       ROUND(AVG(consumo_kwh), 2) AS consumo_medio_kwh,
       ROUND(SUM(consumo_kwh) / SUM(producao_unidades), 4) AS kwh_por_unidade
FROM consumo
GROUP BY turno
ORDER BY kwh_por_unidade
""").show()
```

```
+-----+---------+-----------------+---------------+
|turno|registros|consumo_medio_kwh|kwh_por_unidade|
+-----+---------+-----------------+---------------+
|noite|     1666|           401.66|         0.2116|
|tarde|     1667|           396.71|         0.2123|
|manha|     1667|           399.25|         0.2124|
+-----+---------+-----------------+---------------+
```

Exemplo Indústria consumo energético com Apache Spark

O comando createOrReplaceTempView() cria uma visão temporária (Temporary View) chamada consumo, utilizando os dados do DataFrame df.

Isso permite executar consultas SQL sobre o DataFrame.

- df: DataFrame que contém os registros industriais.
- createOrReplaceTempView(): cria uma visão temporária ou substitui uma existente com o mesmo nome.
- "consumo": nome utilizado para consultar os dados por SQL.

Importante: não cria uma tabela física no banco de dados. A visão fica disponível durante a sessão do Spark.

Exemplo Indústria consumo energético com Apache Spark

O comando SQL realiza uma análise semelhante à feita com groupBy().

- SELECT: define as informações desejadas.
- COUNT(*): conta os registros.
- AVG(): calcula a média.
- SUM(): calcula os totais.
- GROUP BY: organiza os dados em grupos.
- ORDER BY: ordena os resultados.

Exemplo Indústria consumo energético com Apache Spark

8. Indicadores mensais

[8] ✓ 1s

```
mensal = (df.groupBy("ano_mes")
 .agg(F.round(F.sum("consumo_kwh"), 2).alias("consumo_total_kwh"),
      F.sum("producao_unidades").alias("producao_total"))
 .withColumn("kwh_por_unidade", F.round(F.col("consumo_total_kwh")/F.col("producao_total"),4))
 .orderBy("ano_mes"))
mensal.show(12)
```

```
+-------+-----------------+--------------+---------------+
|ano_mes|consumo_total_kwh|producao_total|kwh_por_unidade|
+-------+-----------------+--------------+---------------+
|2026-01|         38687.02|        182271|         0.2123|
|2026-02|         31542.72|        145949|         0.2161|
|2026-03|         38415.62|        178685|          0.215|
|2026-04|         31281.81|        145592|         0.2149|
|2026-05|         39306.31|        185504|         0.2119|
|2026-06|         34109.74|        161039|         0.2118|
|2026-07|         36911.73|        175361|         0.2105|
|2026-08|         35717.66|        170037|         0.2101|
|2026-09|         36311.94|        173051|         0.2098|
|2026-10|          36324.7|        171836|         0.2114|
|2026-11|         35129.55|        165667|          0.212|
|2026-12|          38097.5|        179140|         0.2127|
+-------+-----------------+--------------+---------------+
only showing top 12 rows
```

Exemplo Indústria consumo energético com Apache Spark

9. Temperatura e consumo

Correlação não implica causalidade; demanda de produção, horas e número de máquinas também podem influenciar o consumo.

[9] ✓ 1s

```
1 corr = df.stat.corr("temperatura_ambiente_c", "consumo_kwh")
2 print(f"Correlação de Pearson (temperatura vs. consumo): {corr:.3f}")
3 df.groupBy("maquinas_ativas").agg(F.round(F.avg("consumo_kwh"),2).alias("media_consumo_kwh")).orderBy("maquinas_ativas").show(20)
```

```
Correlação de Pearson (temperatura vs. consumo): 0.011
+---------------+-----------------+
|maquinas_ativas|media_consumo_kwh|
+---------------+-----------------+
|              3|           148.35|
|              4|           190.59|
|              5|            231.5|
|              6|           275.37|
|              7|           315.19|
|              8|           358.53|
|              9|           401.44|
|             10|           450.36|
|             11|           483.03|
|             12|           532.91|
|             13|           560.46|
|             14|            610.1|
|             15|           650.89|
+---------------+-----------------+
```

Exemplo Indústria consumo energético com Apache Spark

```python
import matplotlib.pyplot as plt

# Selecionar as colunas e converter para Pandas
grafico = por_turno.select(
    "turno", "kwh_por_unidade"
).toPandas()

# Criar a figura
plt.figure(figsize=(7,4))

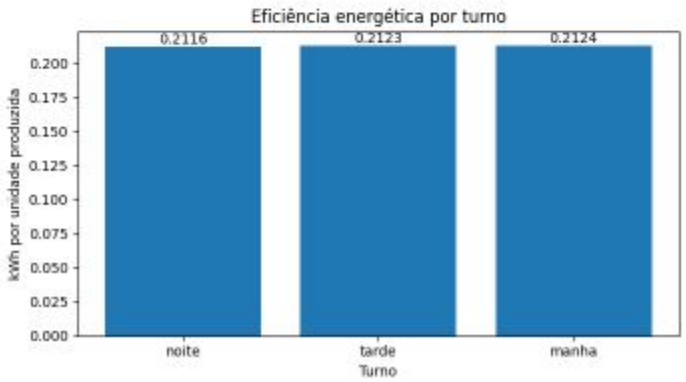
# Criar as barras
barras = plt.bar(
    grafico["turno"],
    grafico["kwh_por_unidade"]
)

# Adicionar os valores em cima das barras
plt.bar_label(barras, fmt="%.4f")

# Identificar os eixos
plt.ylabel("kWh por unidade produzida")
plt.xlabel("Turno")

# Definir o título
plt.title("Eficiência energética por turno")

# Ajustar e exibir
plt.tight_layout()
plt.show()
```



Exemplo Indústria consumo energético com Apache Spark

.toPandas()

Converte o DataFrame do Spark para um DataFrame do Pandas.

Essa conversão permite utilizar os dados facilmente na biblioteca Matplotlib.

PySpark	Pandas
Processamento distribuído	Processamento local
Adequado para grandes volumes	Adequado para dados que cabem na memória
DataFrame Spark	DataFrame Pandas

Exemplo Indústria consumo energético com Apache Spark

Por que converter para Pandas?

Porque o Matplotlib trabalha facilmente com dados de um DataFrame Pandas.

Como o DataFrame por_turno possui apenas os resultados agregados por turno, a conversão é adequada.

Atenção: toPandas() transfere todos os dados selecionados para a memória do computador que executa o código. Não é recomendado converter milhões de registros sem necessidade.

Exemplo Indústria consumo energético com Apache Spark

1. Qual turno possui maior consumo médio?

Utilizar AVG(consumo_kwh) agrupando por turno. O maior valor representa o turno com maior consumo médio por registro.

2. Qual turno consome menos energia por unidade produzida?

Calcular SUM(consumo_kwh) / SUM(producao_unidades). O menor resultado indica maior eficiência energética, em condições comparáveis.

3. Como variam consumo e temperatura?

Utilizar gráficos de linha para analisar a evolução das variáveis ao longo do tempo ou um gráfico de dispersão para investigar sua relação.

4. Como calcular indicadores por mês?

Extrair o mês da coluna de data e utilizar GROUP BY para calcular médias, totais e indicadores mensais.

5. Por que o Spark permite escalar o processamento?

Porque utiliza processamento paralelo e distribuído, dividindo tarefas e dados entre diferentes núcleos ou computadores de um cluster.

Link material

Material disciplina

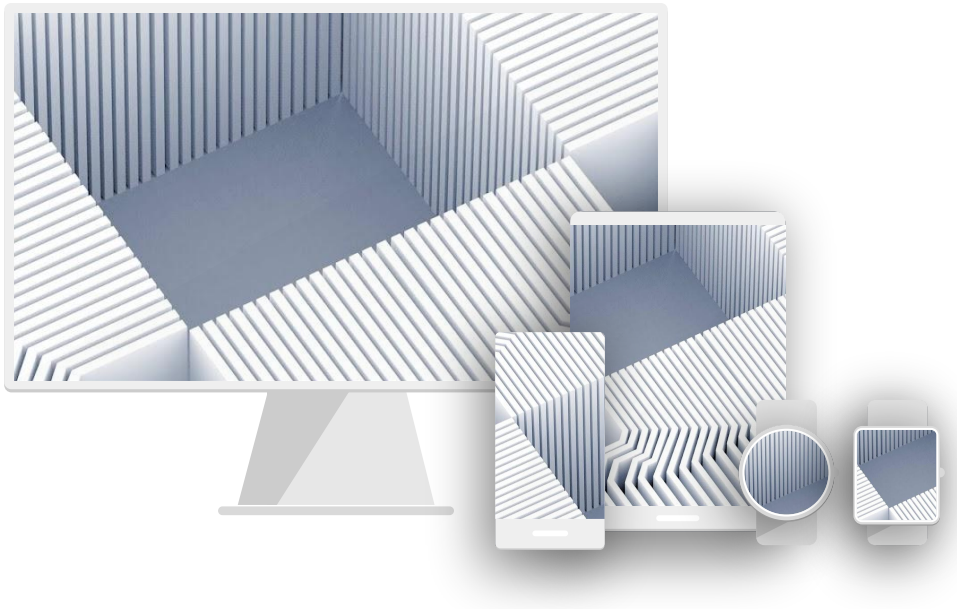
Obrigado !

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95



UNISENAI
Inovação no
Ensino Superior
para a Indústria